GitHub Collaboration & Containerization

Main Features

GitHub builds on Git by adding a web interface for collaboration

- Pull Requests for code review
- Issues for tracking work
- Actions for automation (CI/CD)
- Forking
- Project boards for organization
- Codespaces

Pull Requests

A pull request (PR) is a request to merge your branch into another branch. On GitHub, PRs provide a web interface for code review with line-by-line comments, automated testing through GitHub Actions, discussion threads for design decisions, and approval workflows before merging.

Example data science PR workflow: you develop a new feature extraction method in a branch, open a PR with description and example results, teammates review the code and suggest improvements, automated tests verify the code works, and after approval the PR merges to main.

PRs make code review asynchronous and documented.

Issues

GitHub Issues provide a lightweight way to track work, bugs, and ideas and are ideal for project management

Issues are tickets that describe work to be done. They can track bugs (model not converging), features (add new data source), questions (which algorithm to use), or documentation needs (explain preprocessing steps).

Issues support markdown formatting, can be assigned to team members, can be labeled (bug, enhancement, data), can reference commits and PRs, and can be organized in project boards.

Example: Issue #42 "Add weather data to model" gets assigned to you, you create branch feature/weather-data, make changes, open PR that references "Closes #42", and when PR merges the issue automatically closes.

Actions

GitHub Actions automate workflows like testing, building, and deploying your code.

Actions are triggered by events like pushing code, opening a PR, or on a schedule. Common data science workflows include running tests on every PR, training models on a schedule, generating reports automatically, and deploying models to production.

Example workflow: every time you push code, GitHub Actions runs your unit tests, checks code style with ruff, builds a Docker image, and deploys to staging if tests pass.

This catches bugs early and ensures code quality without manual work.

Forking and Contributing

Forking is how you contribute to projects you don't own, which is essential for open source.

The fork workflow on GitHub: click "Fork" button to create your copy, clone your fork locally, make changes in a branch, push to your fork, then open a PR from your fork to the original repository.

This is how you'll submit assignments: fork the course repo, complete the assignment in your fork, open a PR to submit.

It's also how you contribute to open source data science tools: found a bug in pandas? Fork it, fix it, submit a PR.

Project Boards

Project boards help organize and track work across your team.

Project boards are like Trello or Kanban boards built into GitHub. They provide visual project management with columns (To Do, In Progress, Done), cards that link to issues and PRs, automation (move cards when PRs merge), and team coordination.

For data science projects: track data collection tasks, organize modeling experiments, coordinate team work, and plan feature development.

Example: create a board for your final project with columns for Data, Models, Analysis, and Documentation. Each task becomes a card linked to an issue.

Codespaces

GitHub Codespaces provides a cloud development environment in your browser.

Codespaces is a full VS Code environment that runs in the cloud. You can code from any computer without installing anything, environments are defined in code (devcontainer.json), and you get consistent setup across team members.

For data science: Codespaces can include Python, Jupyter, and all your dependencies pre-installed. You can even access GPUs for model training.

This solves the "works on my machine" problem by giving everyone the same environment.

Best Practices

Following GitHub conventions makes collaboration smoother.

Commit messages: use present tense ("Add feature" not "Added feature"), be specific ("Fix bug in data loader" not "Fix bug"), reference issues when relevant ("Fix #42: Handle missing values").

Branch names: use descriptive names (feature/weather-data not my-branch), follow team conventions (feature/, bugfix/, docs/), and keep branches short-lived (merge within a week).

PR descriptions: explain what and why, show example outputs for data science changes, link to relevant issues, and describe how you tested the changes.

Containerization

Why Containers Matter

Containers solve the reproducibility crisis in data science.

The problem: you train a model on your laptop with Python 3.10 and scikit-learn 1.2. Your teammate tries to run it with Python 3.11 and scikit-learn 1.3. Results are different. Which is correct?

Containers solve this by packaging your exact environment: specific Python version, exact package versions, system dependencies, and environment variables. Your container runs identically on your laptop, your teammate's laptop, and in production.

Container Architecture

A container is a process that runs in an isolated environment. It has its own filesystem (from an image), its own network interface, its own process tree, but shares the host kernel.

An image is a read-only template containing your application and dependencies. Images are built in layers: base OS layer, Python layer, your code layer. Layers are cached and reused, making builds fast.

A container is a running instance of an image. You can run multiple containers from the same image. When a container stops, changes are lost unless you use volumes.

What is Docker?

Docker revolutionized software deployment by making containers mainstream.

Docker is a platform for building, shipping, and running applications in containers. Released in 2013, it made containers accessible to developers by providing simple commands and a huge ecosystem of pre-built images.

Key Docker concepts:

- Images are templates (like a recipe)
- Containers are running instances (like a dish made from the recipe)
- Dockerfile is the recipe file
- Docker Hub is a registry of shared images.

Why Docker Became Popular

Docker solved real problems that developers faced daily.

Before Docker: "It works on my machine" was a common problem. Developers spent hours setting up environments. Deploying applications required complex configuration. Different environments (dev, staging, production) had subtle differences causing bugs.

Docker's solutions: Consistent environments across all machines, Fast setup (pull an image, run it), Easy deployment (ship the container), Isolation (no conflicts between projects).

Docker meant reproducible analysis, easy collaboration, and simple deployment of models.

Docker Architecture and Ecosystem

Understanding Docker's architecture helps you use it effectively.

Docker architecture: Docker daemon runs as a service (background process), Docker CLI sends commands to the daemon, Images are stored locally and pulled from registries, Containers run as isolated processes.

Docker ecosystem: Docker Hub hosts millions of images (python, postgres, jupyter, etc.), Official images are maintained and trusted, Community images provide specialized tools, Docker Compose manages multi-container apps.

For data science: The ecosystem is huge. Need Python? Pull python:3.11. Need Jupyter? Pull jupyter/scipy-notebook. Need PostgreSQL? Pull postgres. Everything is pre-built and ready to use.

Docker vs Podman

Podman is a drop-in replacement for Docker with better security.

Docker requires a daemon running as root, which is a security risk. Podman runs containers as your user (rootless), has no daemon (each command is independent), and uses the same command syntax as Docker.

For this course we use Podman because it's more secure and works better on macOS. Most Docker tutorials work with Podman by replacing 'docker' with 'podman'.

Key difference: Docker uses docker-compose, Podman uses podman-compose or Kubernetes YAML.

Installing Podman

Podman installation varies by operating system but is straightforward.

MacOS: `brew install podman`, then `podman machine init` and `podman machine start` (Podman runs in a VM on macOS).

Linux: Use your package manager - `apt install podman` (Ubuntu/Debian) or `dnf install podman` (Fedora/RHEL). Podman runs natively on Linux.

Windows: Install via WSL2 (Windows Subsystem for Linux), then follow Linux instructions.

Verify installation: `podman --version` and `podman run hello-world`

podman

Podman Setup and Configuration

After installation, some configuration improves the experience.

For macOS users: `podman machine set --rootful=false` ensures rootless mode, `podman machine set --cpus 4 --memory 8192` allocates resources.

Useful aliases: Add to your .bashrc or .zshrc: alias docker=podman (makes Docker tutorials work), alias docker-compose=podman-compose.

Test your setup: podman run -it python:3.11 python (should drop you into a Python REPL), podman ps (should show running containers), podman images (should show downloaded images).

If you encounter issues, check podman machine status and restart with podman machine stop && podman machine start.

Your First Container

Let's run a simple container to understand the basics.

Command: `podman run -it python:3.11 python`

This downloads the python:3.11 image (if not cached), starts a container from that image, runs the python command inside, and connects your terminal to it (-it flags).

You're now in a Python REPL inside a container. Try:

```
import sys; print(sys.version).
```

Exit with exit(). The container stops when the command finishes.

This is the foundation: run a command in an isolated environment.

Running Jupyter in a Container

Jupyter notebooks are perfect for containerization.

Command: `podman run -p 8888:8888 jupyter/scipy-notebook`

This runs a container with Jupyter, scipy, pandas, matplotlib pre-installed. The -p 8888:8888 flag maps port 8888 in the container to port 8888 on your laptop.

Open the URL shown in the terminal. You're now using Jupyter running in a container. Create a notebook, import pandas, load data. When you stop the container, the notebook is lost (unless you use volumes).

This is on e way to try new tools without installing them.

Container Volumes for Data

Volumes let containers access data on your computer.

Problem: containers are ephemeral. When they stop, changes are lost.

Solution: mount a directory from your computer into the container.

Command: `podman run -v $(pwd)/data:/data -it python:3.11 bash`

This mounts your local data/ directory to /data inside the container. Files in /data persist after the container stops. You can read data files, write results, and they're saved on your computer.

For data science: mount your data directory and your code directory.

Building Your First Image

Images are built from Dockerfiles that specify the environment.
A Dockerfile is a recipe for building an image. Example for data science:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "train.py"]
```

Build: podman build -t my-model .
Run: podman run my-model

Dockerfile Best Practices

Writing good Dockerfiles makes builds fast and images small.

- Use specific base image versions (python:3.11-slim not python:latest).
- Install dependencies before copying code (enables caching).
- Combine RUN commands to reduce layers.
- Use .dockerignore to exclude unnecessary files.
- Don't run as root (USER statement).

Example optimization: instead of RUN pip install pandas, RUN pip install numpy, use RUN pip install pandas numpy. This creates one layer instead of two.

Small images build faster, push faster, and are more secure.

Data Science Container Example

Let's build a complete data science container.
Dockerfile:

```
FROM python:3.11-slim
RUN apt-get update && apt-get install -y git
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ ./src/
COPY data/ ./data/
CMD ["python", "src/train.py"]
```

Build: podman build -t sentiment-model .
Run: podman run -v $(pwd)/results:/app/results sentiment-model

Results are saved to your local results/ directory.

Serving Models with Containers

Containers are a standard way to deploy ML models.

You train a model, save the weights, build a container with a web server that loads the model and serves predictions. Deploy the container to production.

Example: FastAPI app that loads a scikit-learn model and serves predictions via HTTP. The container includes Python, FastAPI, scikit-learn, and your model weights.

Command: podman run -p 8000:8000 my-model-api

Now you can send HTTP requests to localhost:8000/predict.

Multi-Stage Builds

Multi-stage builds create smaller, more secure images.

Problem: you need build tools (compilers, dev headers) to install packages, but you don't need them at runtime. They bloat your image.

Solution: use multiple FROM statements. First stage installs everything and builds. Second stage copies only the built artifacts.

Example: first stage compiles numpy from source (needs gcc, build tools). Second stage copies the compiled numpy (doesn't need build tools). Final image is much smaller.

This is especially useful for data science packages with C extensions.

Container Networking

Containers can communicate with each other through networks.

By default, containers are isolated. To connect them, create a network: podman network create ml-network

Run containers on the network: podman run --network ml-network --name db postgres, podman run --network ml-network --name api my-api

The api container can now connect to the db container using hostname 'db'. This is how you build multi-container applications.

Example: database container + API container + model training container.

Docker Compose for Multi-Container Apps

Docker Compose (or podman-compose) manages multiple containers.

Instead of running multiple podman run commands, define all containers in a YAML file:

```
services:
  db:
    image: postgres
  jupyter:
    build: .
    ports:
      - "8888:8888"
    volumes:
      - ./data:/data
```

Start everything: podman-compose up

This is how you define reproducible multi-container environments.

GPU Support in Containers

Containers can access GPUs for deep learning.

To use GPUs in containers, you need the NVIDIA Container Toolkit installed. Then run:

```
podman run --device nvidia.com/gpu=all pytorch/pytorch python train.py
```

The container can now access your GPU. This is how you train deep learning models in containers.

Benefits: reproducible GPU environment, easy to switch between GPU and CPU, can run multiple GPU jobs in separate containers.

Essential for deep learning in production.

Container Security Best Practices

Containers improve security but require best practices.

Don't run as root: add USER statement to Dockerfile. Use official base images from trusted sources. Keep images updated (rebuild regularly). Scan images for vulnerabilities (podman scan). Don't include secrets in images (use environment variables or secrets management).

Example bad practice: COPY .env . in Dockerfile (secrets in image). Good practice: pass secrets as environment variables at runtime.

Security matters for production data science systems.

Debugging Containers

Debugging containers requires different techniques than debugging local code.

View logs: podman logs container-name. Execute commands in running container: podman exec -it container-name bash. Inspect container: podman inspect container-name.

Common issues: container exits immediately (check logs), can't connect to service (check port mapping), file not found (check volume mounts), permission denied (check user/permissions).

Pro tip: add a sleep infinity command to keep container running while you debug.

Container Registries

Registries store and distribute container images.

Docker Hub is the default registry (docker.io). GitHub Container Registry (ghcr.io) integrates with GitHub. Cloud providers have their own registries (AWS ECR, Google GCR, Azure ACR).

Push image: podman tag my-model ghcr.io/username/my-model, podman push ghcr.io/username/my-model

Pull image: podman pull ghcr.io/username/my-model

This is how you share images with teammates and deploy to production.

Containers in Production

Deploying containers to production requires orchestration.

For simple deployments: run containers on a VM with systemd or docker-compose. For complex deployments: use Kubernetes for orchestration, auto-scaling, load balancing, and self-healing.

Cloud platforms make this easy: AWS ECS, Google Cloud Run, Azure Container Instances. They handle orchestration, scaling, and monitoring.

Example: push your model container to a registry, deploy to Cloud Run, get a URL for predictions. Cloud Run auto-scales based on traffic.

Containers for Data Pipelines

Containers are perfect for data processing pipelines.

Each step of your pipeline runs in a container: data ingestion container, data cleaning container, feature engineering container, model training container, model evaluation container.

Benefits: each step is isolated and reproducible, steps can run in parallel, easy to retry failed steps, can run on different machines.

Tools like Airflow or Prefect orchestrate container-based pipelines. This is how production data science works.

Containers vs Virtual Environments

Containers and virtual environments solve different problems.

Virtual environments (venv, conda) isolate Python packages. Containers isolate the entire system (OS, Python, packages, system libraries).

Use virtual environments for: local development, quick experiments, when you only need Python isolation.

Use containers for: reproducibility across machines, deployment, when you need system dependencies, when you need isolation beyond Python.

Best practice: **use both**. Develop in a virtual environment, deploy in a container.

Modern Python Package Management with uv

The Python Packaging Problem

Python package management has been painful for years.

pip is slow (resolves dependencies one at a time), doesn't lock versions by default (pip freeze is manual), doesn't handle conflicts well, and doesn't manage Python versions.

conda is better but slow, creates huge environments, and has its own ecosystem separate from PyPI.

poetry and pipenv improved things but are still slow and have their own issues.

What is uv

uv is a fast Python package manager written in Rust.

Created by Astral (makers of ruff), uv is 10-100x faster than pip, handles dependency resolution correctly, creates lock files automatically, manages Python versions, and is compatible with pip and PyPI.

Key insight: uv is a drop-in replacement for pip. Commands like pip install become uv pip install. But uv is much faster and smarter.

For data science: faster package installation means faster iteration.

Installing uv

Getting started with uv is simple.

On macOS/Linux: `curl -LsSf https://astral.sh/uv/install.sh | sh`

On Windows: `powershell -c "irm https://astral.sh/uv/install.ps1 | iex"`

Or with pip: pip install uv (though this defeats the speed purpose)

After installation, verify: `uv --version`

uv Speed Comparison

The speed difference is dramatic.

Benchmark: install pandas, numpy, scikit-learn, matplotlib, seaborn (common data science stack).

pip: 45 seconds. conda: 2 minutes. poetry: 1 minute. uv: 4 seconds.

Why so fast? uv uses parallel downloads, caches aggressively, resolves dependencies in Rust (compiled, not interpreted), and uses a better dependency resolution algorithm.

Creating Virtual Environments with uv

uv makes virtual environments easier.

Old way: python -m venv .venv, source .venv/bin/activate, pip install packages

New way: uv venv, source .venv/bin/activate, uv pip install packages

Or even simpler: uv venv creates the environment, uv pip install --system packages (no activation needed)

uv also manages Python versions: uv venv --python 3.11 creates an environment with Python 3.11 even if you don't have it installed.

Dependency Management with uv

uv handles dependencies better than pip.

Command: uv pip compile requirements.in -o requirements.txt

This resolves all dependencies and creates a lock file with exact versions. Your requirements.in has high-level dependencies (pandas, scikit-learn). Your requirements.txt has every dependency with exact versions (pandas==2.1.0, numpy==1.24.3, etc).

Install from lock file: uv pip sync requirements.txt

This ensures everyone has identical environments.

uv for Data Science Projects

A typical data science project workflow with uv.

Create project: mkdir my-project && cd my-project

Create environment: uv venv

Create requirements.in with your dependencies: pandas, scikit-learn, jupyter

Compile lock file: uv pip compile requirements.in -o requirements.txt

Install: uv pip sync requirements.txt

Now you have a reproducible environment. Commit both requirements.in and requirements.txt to Git.

uv in Containers

uv makes container builds faster.

Dockerfile with pip:

```
FROM python:3.11-slim
COPY requirements.txt .
RUN pip install -r requirements.txt
```

(takes 45 seconds)

Dockerfile with uv:

```
FROM python:3.11-slim
COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv
COPY requirements.txt .
RUN uv pip install --system -r requirements.txt
```

(takes 4 seconds)

Faster builds mean faster iteration.

uv and GitHub Actions

uv speeds up CI/CD pipelines.

GitHub Actions workflow with pip:

- run: pip install -r requirements.txt

(takes 2 minutes)

GitHub Actions workflow with uv:

- run: curl -LsSf https://astral.sh/uv/install.sh | sh
- run: uv pip install --system -r requirements.txt

(takes 20 seconds)

Faster CI means faster feedback on PRs.

uv Best Practices

Follow these practices for effective uv usage.

Always use lock files (requirements.txt from uv pip compile). Commit both requirements.in and requirements.txt. Use --system in containers and CI. Use specific Python versions (uv venv --python 3.11). Update dependencies regularly (uv pip compile --upgrade).

Don't mix pip and uv in the same environment. Don't use uv pip freeze (use uv pip compile instead). Don't skip the lock file (defeats reproducibility).

Migrating from pip to uv

Migrating existing projects to uv is straightforward.

If you have requirements.txt: rename to requirements.in, run uv pip compile requirements.in -o requirements.txt, install with uv pip sync requirements.txt.

If you have setup.py or pyproject.toml: uv can read these directly. Run uv pip install -e . to install in editable mode.

If you have conda: export environment to requirements.txt, then follow the process above.

Migration takes 5 minutes and makes your project faster and more reproducible.

uv Limitations and Alternatives

uv is great but not perfect for every use case.

uv doesn't handle non-Python dependencies (use conda or containers). uv doesn't have built-in project management like poetry (just package management). uv is new (less mature than pip).

When to use conda instead: you need non-Python packages (R, system libraries), you need conda-forge packages, you're in a conda-heavy organization.

When to use poetry instead: you're publishing packages to PyPI, you want integrated project management.

For most data science work, uv is the best choice.

Putting It All Together

Reproducible Data Science Workflow

Combine GitHub, containers, and uv for reproducible data science.

Your project structure: code in GitHub with branches and PRs, dependencies managed with uv (requirements.in and requirements.txt), environment defined in Dockerfile, data and results in mounted volumes.

Workflow: develop locally with uv venv, test in container with podman build, push to GitHub with PR, CI runs tests in container with uv, merge to main, deploy container to production.

Example: Complete Data Science Project

Let's walk through a complete project: LA Crime Analysis.

Project: Analyze and predict crime patterns in Los Angeles using real data from the LA Open Data Portal. GitHub repo with main branch, feature branches for new models, requirements.in with dependencies, Dockerfile for reproducible environment, docker-compose.yml for Jupyter + API services, data mounted as volumes.

Development workflow: create feature branch for new analysis, develop in Jupyter container, commit changes with clear messages, open PR with results and visualizations. CI runs: GitHub Actions builds container with uv, runs tests, checks code quality. Review: teammate reviews code and methodology, suggests improvements. Merge: PR merges to main after approval. Deploy: container deploys to staging for testing.

Template repository: github.com/[instructor-username]/la-crime-analysis - students can fork this for their final projects. It demonstrates all concepts from this week in a real, working project.

Common Pitfalls and Solutions

Avoid these common mistakes.

Pitfall: not using lock files. Solution: always use uv pip compile.
Pitfall: not testing in containers. Solution: test locally in containers before pushing.
Pitfall: large container images.
Solution: use multi-stage builds and slim base images. Pitfall: secrets in containers.
Solution: use environment variables.
Pitfall: not using branches. Solution: always develop in feature branches.

These practices prevent problems before they occur.

Container Example: Web Scraping Pipeline

Containers are ideal for web scraping projects.

Dockerfile for scraper:

```
FROM python:3.11-slim
RUN apt-get update && apt-get install -y chromium chromium-driver
WORKDIR /app
COPY requirements.txt .
RUN uv pip install --system -r requirements.txt
COPY scraper.py .
CMD ["python", "scraper.py"]
```

Run: podman run -v $(pwd)/data:/app/data scraper

The container includes Chrome for JavaScript-heavy sites. Data is saved to your local data/ directory. This ensures the scraper works identically on any machine.

Container Example: Database for Analysis

Run a database in a container for data analysis.

Start PostgreSQL: podman run -d --name postgres -e POSTGRES_PASSWORD=secret -p 5432:5432 -v $(pwd)/data:/var/lib/postgresql/data postgres

Connect from Python: import psycopg2; conn = psycopg2.connect(host='localhost', port=5432, user='postgres', password='secret')

Load data, run queries, analyze results. When done: podman stop postgres. Data persists in the volume.

This is faster than installing PostgreSQL locally and easier to clean up.

Container Example: API for Data Collection

Build an API container to collect data from external sources.

Your API fetches data from Twitter, weather services, stock APIs, etc. The container includes your API keys (as environment variables), rate limiting logic, and data storage.

Dockerfile:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN uv pip install --system -r requirements.txt
COPY api_collector.py .
CMD ["python", "api_collector.py"]
```

Run: `podman run -e TWITTER_KEY=$TWITTER_KEY -v $(pwd)/collected:/app/data api-collector`

Data collection runs in the background, saving to your local directory.

Container Example: Batch Processing

Process large datasets in batches using containers.

Your script processes data files one at a time. Run multiple containers in parallel to process faster:

```
podman run -v $(pwd)/data:/data processor process file1.csv
podman run -v $(pwd)/data:/data processor process file2.csv
podman run -v $(pwd)/data:/data processor process file3.csv
```

Each container processes one file independently. This parallelizes your work without complex code.

For very large datasets, this can reduce processing time from hours to minutes.

Container Example: Reproducible Notebooks

Share Jupyter notebooks with guaranteed reproducibility.

Dockerfile:

```
FROM jupyter/scipy-notebook
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY notebooks/ /home/jovyan/work/
```

Build: `podman build -t my-analysis .`

Run: `podman run -p 8888:8888 my-analysis`

Anyone can run your analysis with identical results. The notebook, data, and environment are all in the container.

This is how you share reproducible research.

GitHub Actions Example: Automated Testing

Set up automated testing for your data science project.

.github/workflows/test.yml:

```
name: Test
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: curl -LsSf https://astral.sh/uv/install.sh | sh
      - run: uv venv && uv pip sync requirements.txt
      - run: source .venv/bin/activate && pytest
```

This runs your tests on every push and PR. Catches bugs before they reach main.

GitHub Actions Example: Automated Reports

Generate reports automatically on a schedule.

.github/workflows/report.yml:

```yaml
name: Weekly Report
on:
  schedule:
    - cron: '0 0 * * 0'
jobs:
  report:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: python generate_report.py
      - uses: actions/upload-artifact@v3
        with:
          name: report
          path: report.html
```

This generates a report every Sunday. Useful for monitoring data quality, model performance, or business metrics.

GitHub Actions Example: Model Training

Train models automatically when data changes.

.github/workflows/train.yml:

```yaml
name: Train Model
on:
  push:
    paths:
      - 'data/**'
jobs:
  train:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: podman build -t model .
      - run: podman run -v $(pwd)/models:/app/models model
      - uses: actions/upload-artifact@v3
        with:
          name: model
          path: models/
```

When you push new data, the model retrains automatically. This is continuous training.

Debugging Tips: Container Issues

Common container issues and solutions.

Issue: container exits immediately. Solution: check logs (podman logs), check CMD in Dockerfile, try running with bash (podman run -it image bash).

Issue: can't access service. Solution: check port mapping (-p flag), check if service is listening (podman exec container netstat -tlnp).

Issue: file not found. Solution: check volume mount (-v flag), check paths in container (podman exec container ls /path).

Issue: permission denied. Solution: check file permissions, check USER in Dockerfile, try running as root (--user root).

Debugging Tips: uv Issues

Common uv issues and solutions.

Issue: package not found. Solution: check package name on PyPI, check if you need a different index (--index-url).

Issue: dependency conflict. Solution: check requirements.in for conflicting versions, use uv pip compile --upgrade to resolve.

Issue: slow installation. Solution: check if cache is working (uv cache dir), clear cache if needed (uv cache clean).

Issue: wrong Python version. Solution: specify version explicitly (uv venv --python 3.11), check installed versions (uv python list).

Performance Tips: Container Optimization

Make your containers faster and smaller.

Use slim base images (python:3.11-slim not python:3.11). Use multi-stage builds for compiled dependencies. Combine RUN commands to reduce layers. Use .dockerignore to exclude unnecessary files. Cache dependencies separately from code.

Example optimization:

Before: 2GB image, 5 minute build

After: 500MB image, 1 minute build

Smaller images build faster, push faster, and start faster.

Performance Tips: uv Optimization

Make uv even faster.

Use lock files (uv pip compile) to skip dependency resolution. Use --system in containers to skip venv overhead. Use uv cache to reuse downloads. Use --no-deps when you know dependencies are satisfied.

Example: installing 50 packages with lock file takes 5 seconds. Without lock file takes 30 seconds.

Lock files are the key to fast, reproducible installs.